

Project Design Phase Solution Architecture

Date	01 October 2026
Team ID	6ab7f31f8c7c66e70672939b
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	4 Marks

Solution Architecture:

Solution architecture is a complex process – with many sub-processes – that bridges the gap between business problems and technology solutions. Its goals are to:

- Find the best tech solution to solve existing business problems.
- Describe the structure, characteristics, behavior, and other aspects of the software to project stakeholders.
- Define features, development phases, and solution requirements.
- Provide specifications according to which the solution is defined, managed, and delivered.

Solution Architecture Diagram:

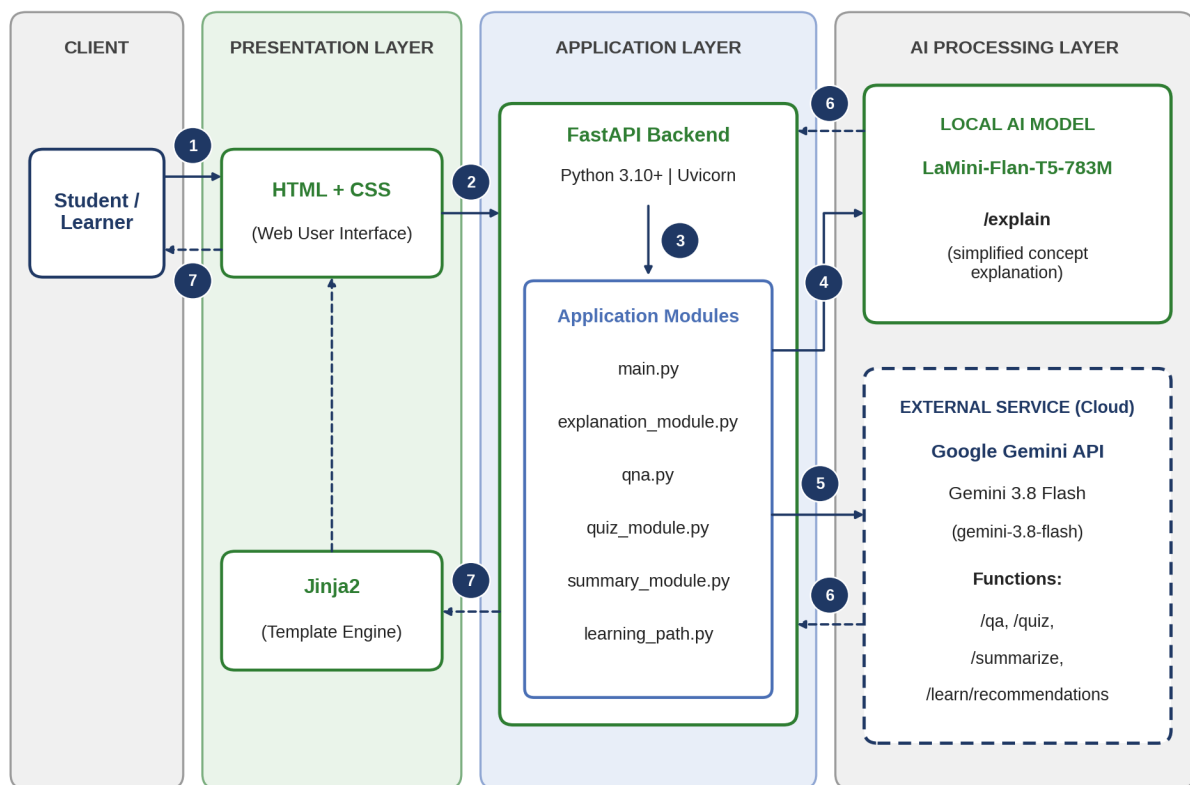


Figure 1: Architecture and data flow of the EduGenie learning assistant prototype

Data flow: 1 Student submits a request in the web interface. 2 The UI forwards it to the FastAPI backend. 3 FastAPI routes it to the matching feature module. 4 /explain is handled by the local LaMini-Flan-T5-783M model. 5 /qa, /quiz, /summarize and /learn/recommendations are sent to the Gemini API (gemini-3.8-flash). 6 The AI output returns to FastAPI. 7 The response is rendered through Jinja2 and HTML and shown to the student. No database or cloud storage is used in the prototype.

Reference: EduGenie Project Design Phase-II: Technology Stack (Architecture & Stack).

Architecture Overview:

EduGenie follows a layered solution architecture consisting of the Client Presentation Layer, the Application Layer and the AI Processing Layer. The Client Presentation Layer provides the student-facing web interface using HTML, CSS and Jinja2. The Application Layer uses FastAPI with Python and Uvicorn to receive requests and route them to the appropriate learning module. The AI Processing Layer combines the locally hosted LaMini-Flan-T5-783M model for simplified concept explanation with the external Google Gemini API, using Gemini 3.8 Flash, for academic Q&A, quiz generation, content summarization and learning recommendations. The generated response is returned through the FastAPI backend and rendered to the student through Jinja2 and HTML. The current prototype does not use a database or cloud storage.

Mapping of Student Problems to the Architecture:

Student Problem	Application Modules	AI Used	Student Output
Problem 1 Difficulty understanding complex academic concepts	explanation_module.py (/explain) qna.py (/qa)	LaMini-Flan-T5-783M Gemini 3.8 Flash	Simplified explanation and academic answer
Problem 2 Difficulty following a structured learning progression	learning_path.py (/learn/recommendations)	Gemini 3.8 Flash	Learning recommendations / structured progression
Problem 3 Difficulty efficiently reviewing and practising learning content	summary_module.py (/summarize) quiz_module.py (/quiz)	Gemini 3.8 Flash	Concise summary and practice quiz

AI Model Mapping:

Function	Endpoint	AI
Concept Explanation	/explain	LaMini-Flan-T5-783M (local)
Academic Q&A	/qa	Gemini 3.8 Flash (external)
Quiz Generation	/quiz	Gemini 3.8 Flash (external)
Summarization	/summarize	Gemini 3.8 Flash (external)
Learning Recommendations	/learn/recommendations	Gemini 3.8 Flash (external)

Architecture Characteristics:

Modularity: the major learning functions (explanation, Q&A, quiz, summarization and learning path) are separated into dedicated modules, which makes the structure easier to maintain and extend.

Model separation: local AI processing (LaMini-Flan-T5-783M) is separated from external generative AI processing (Gemini 3.8 Flash).

Lightweight web architecture: HTML, CSS, Jinja2, FastAPI, Python and Uvicorn are used instead of a heavy frontend framework.

External AI integration: Gemini-powered functions communicate through the Google Gemini API; the Gemini service is not owned or hosted by EduGenie.

No persistent storage: the current architecture does not include a database or cloud storage component.

Future growth: the modular architecture provides a foundation for future production deployment and scaling.